

Reflex Computation: Automatic Calculator Assistance During Autoregressive Decoding

Working Draft

August 2026

Abstract

Language models can describe arithmetic procedures while still making exact execution errors. Conventional tool-use systems address this weakness by requiring the model to decide to call a calculator and serialize an invocation. We study a narrower alternative: a bounded calculator watches the generated character stream for completed strict arithmetic syntax, normalizes the expression into typed postfix micro-IR, evaluates it exactly, and inserts the result immediately after the completing equals sign. The model need not emit a function name or API schema. This first experiment is intentionally small: it uses integer literals, an allowlisted operator grammar, append-only trace state, and textual reinjection; it includes no learned trigger, general symbolic reasoner, backtracking, or native key/value interface. We define matched model-only, prompted, explicit-tool, reflex, and oracle conditions; a factorial sweep over expression depth, operand width, model size, and seed; component and end-to-end metrics; adversarial non-trigger controls; and precise falsification criteria. The runtime, generator, replay path, optional Hugging Face adapter, and test suite are implemented. At this protocol revision no language-model experiment has been run, so we report no empirical performance claim.

1 Status and Scope

This document is an implemented experimental protocol, not a completed result. The repository contains a dependency-free calculator/runtime path, deterministic dataset generation, JSONL traces and summaries, a scripted plumbing check, and an optional token-by-token Hugging Face adapter. Scripted checks are marked `empirical: false`; their expected perfect calculator behavior must not be cited as model evidence.

Paper 1 asks one question:

When a language model naturally exposes a completed arithmetic expression during generation, does automatic bounded evaluation improve reliability or cost relative to model-only generation and explicit calculator calls?

The scope is calculator only and strict syntax only. The experiment does not test semantic recognition of prose arithmetic, learned routing, persistent symbolic memory, transactional state, Progressive Retrieval Attention (PRA), or native key/value access. Those mechanisms would make a positive result harder to attribute and a negative result harder to diagnose.

2 Hypotheses

H1: exact reliability. On arithmetic examples for which the strict event is correctly detected and normalized, reflex assistance increases exact-match accuracy relative to the same model without assistance.

H2: interface efficiency. Reflex assistance reduces model-authored tool-selection and argument tokens relative to an explicit calculator condition while remaining competitive in answer quality and wall time.

H3: difficulty scaling. As operator count and operand width increase, reflex accuracy degrades more slowly than model-only and matched-prompt conditions. The relevant evidence is the condition-by-difficulty interaction or fitted scaling slope, not one easy benchmark cell.

H4: model-size leverage. Smaller models receive at least as much marginal accuracy benefit from correct reflex assistance as larger matched-family models. This is tested as a model-size-by-condition interaction, not inferred from separate unrelated checkpoints.

These hypotheses are conditional on detection and use. Oracle detection estimates interface headroom; engine correctness alone does not demonstrate that the model will expose the syntax or use the inserted result.

3 Mechanism

3.1 Strict generation-time event

The reflex recognizes an integer arithmetic suffix completed by one equals sign, for example

The total is 37 * 48 =

and inserts the exact result before later generated text:

The total is 37 * 48 = 1776.

The event is ordinary written arithmetic, not a bespoke API token. Detection is incremental at character granularity even when a tokenizer supplies multi-character fragments. Thus, if an equals sign occurs in the middle of a fragment, the result is inserted before the remainder of that fragment is processed.

The lexical detector accepts digits, whitespace, parentheses, and +, -, *, /, //, %, and **. It requires an operator-like symbol, balanced parentheses, and a digit or closing parenthesis before the equals sign. It suppresses double-quoted candidates and rejects candidates adjacent to letters, underscores, or decimal points. Lexical detection remains separate from normalization: a candidate such as + 2 is logged and then rejected because it has no binary operation.

3.2 Typed micro-IR

The normalizer parses the candidate in expression mode and accepts only integer constants, binary operations, unary plus, and unary minus. Calls, names, attributes, indexing, containers, comparisons, booleans, floats, and all other syntax are rejected. The parsed tree is converted to a canonical postfix instruction sequence such as

PUSH 7; PUSH 5; ADD; PUSH 3; MUL.

The Python parser is used only to construct and inspect syntax. The source is never compiled or executed. A separate stack interpreter executes the micro-IR using exact rational numbers.

Table 1: Default calculator safety and complexity bounds.

Bound	Default
Expression characters	256
AST nodes / depth	64 / 16
Digits per integer literal	64
Absolute integer exponent	12
Numerator or denominator size	512 bits
Stack items	64
Engine time budget	25 ms
Recognizer suffix buffer	512 characters

3.3 Resource policy

Every event is bounded before or during evaluation. The default policy is shown in Table 1. Bounds are configuration recorded in each request, not hidden assumptions.

Division by zero, malformed IR, timeout, and resource exhaustion are typed failures. Failed results are traced but not inserted. Exponent and intermediate bit limits are checked before a large value can propagate through subsequent instructions.

3.4 Observable interrupt pipeline

Each intervention follows

DETECT $\rightarrow$ NORMALIZE $\rightarrow$ ROUTE $\rightarrow$ EXECUTE $\rightarrow$ STATE UPDATE $\rightarrow$ REINJECT.

Every transition receives a run ID, sequence number, timestamp, status, candidate/request IDs, duration where meaningful, and structured details. The append-only micro-state stores the typed request, result, creation time, detector, and engine provenance. This is enough to separate detector, parser, engine, and reinjection failures without importing the later transactional-state design.

4 Conditions

All conditions use the same arithmetic examples, checkpoint revision, decoding budget, and calculator implementation when a calculator is available.

The explicit format is deliberately unambiguous and is not counted as a reflex. Its parser and the reflex parser converge on the same arithmetic micro-IR and engine, isolating invocation style from calculator capability. A stronger native function-calling baseline should be added when the selected model/runtime exposes a stable function-calling interface; the textual baseline is the portable minimum.

5 Benchmark

5.1 Factorial arithmetic generator

The deterministic generator crosses operator count with operand digit width and draws operators from addition, subtraction, multiplication, and exact rational division. Expressions are parenthesized left-compositions so the gold parse is unambiguous. Each cell uses identity-derived random

Table 2: Minimum experimental conditions.

Condition	Model instruction and capability	Purpose
LLM only	Return the exact answer; no calculator	Unassisted checkpoint baseline
Matched prompt	Work carefully and check operations; no calculator	Stronger prompting without external capability
Explicit tool	Model writes <code><tool:calculator>EXPR</tool></code> and receives the same engine result	Conventional model-authored invocation baseline
Reflex	Model is asked to write the expression followed by one equals sign; runtime detects any valid strict suffix	Proposed automatic condition
Oracle	Same generation instruction as reflex, but the detector accepts only the gold canonical expression	Detection/selection headroom control

seeds, and gold answers are produced by a separate exact-rational reference evaluator rather than the calculator implementation under test. The primary configuration contains

$$4 \text{ operator counts} \times 4 \text{ digit widths} \times 100 \text{ examples} = 1600$$

arithmetic examples plus 200 non-trigger controls.

This synthetic benchmark isolates scaling and component correctness; it does not establish ecological prevalence. A later extension may add held-out natural model traces, but only after freezing annotation rules for whether a generated prefix contains a valid strict event.

5.2 Non-trigger and adversarial controls

Controls include prose containing numbers, ranges, parentheses, quoted arithmetic, decimal-looking versions, variable equations, and code-like equality. Additional engine tests cover division by zero, excessive exponents and values, disallowed calls/names/indexing, and unary-only candidates. Controls are scored for false intervention and component behavior, not arithmetic answer accuracy.

5.3 Model and seed sweep

The primary manifest pins one matched Qwen3 family at 0.6B, 1.7B, and 4B parameters and uses seeds 17, 29, and 41. Repository commit hashes are stored in the configuration and copied into run manifests. The model list is an initial controlled family, not a claim of model-family generality. A second family is required before making architecture-general claims.

Generation is greedy in the initial adapter so seed variability mainly reflects future sampled configurations and experimental plumbing; if sampling is enabled, temperature, top- p , and all random states must be pinned. The correctness-first adapter recomputes the full prefix per generated token after reinjection. This is intentionally inefficient but semantically transparent. Optimized KV-cache interception is outside Paper 1 and must not be silently credited to it.

6 Metrics and Analysis

End task. Report exact rational answer accuracy with 95% confidence intervals. If a model states a final answer after receiving a calculator result, score the final claim, not merely the correct engine value. Report correction, override, and result-use rates separately.

Components. Report candidate count, trigger precision/recall, false-intervention rate, normalization acceptance/correctness, engine success/correctness, reinjection success, intervention count, and state-item count. Oracle differences localize missed or malformed strict events.

Cost. Report prompt, model-generated, and reinjected tokens separately; model calls; engine and end-to-end wall time; and serialized trace/state growth. Do not present the correctness-first Hugging Face adapter’s full-prefix recomputation as a production latency estimate.

Scaling and inference. Report every operator-count by digit-width cell for each model, condition, and seed. Estimate condition-by-difficulty and condition-by-model-size interactions, with paired comparisons because all conditions share examples. Predeclare one primary endpoint and correct or label exploratory multiple comparisons. Preserve raw predictions and event traces so component errors can be reclassified.

7 Falsification and Evidence Gate

The reflex claim is not supported if any of the following holds under matched capability and declared cost accounting:

- answer gains disappear when final model claims, rather than inserted engine values, are scored;
- false triggers, normalization errors, or ignored results offset exact execution gains;
- explicit calculator calling matches or dominates the quality–token–latency frontier;
- reflex and baseline accuracy have indistinguishable difficulty slopes;
- gains occur only with oracle detection or only in one easy/model cell; or
- results fail to reproduce across seeds or a second model family.

Paper 2 is justified only if Paper 1 identifies a reproducible regime where automatic strict assistance helps and mixed operation types expose a limitation of one calculator. Paper 3 is not justified by syntax brittleness alone; learned interrupts require a measured task regime worth extending.

8 Reproduction

The reusable runtime lives under `src/ccpu/common`; Paper 1 code lives under `src/ccpu/paper1`. From the repository root, the dependency-free protocol check is:

```
python -m pip install -e .
python -m pytest
python -m ccpu paper1 generate \
    --config configs/paper1/smoke.json \
    --output artifacts/paper1/smoke/dataset.jsonl
python -m ccpu paper1 validate \
    --dataset artifacts/paper1/smoke/dataset.jsonl
python -m ccpu paper1 simulate \
    --dataset artifacts/paper1/smoke/dataset.jsonl \
    --output-dir artifacts/paper1/smoke/run
```

The pinned pretrained path is:

```
python -m pip install -e ".[hf]"
python -m ccpu paper1 generate \
    --config configs/paper1/primary.json \
    --output artifacts/paper1/primary/dataset.jsonl
python -m ccpu paper1 run-hf \
    --dataset artifacts/paper1/primary/dataset.jsonl \
    --config configs/paper1/primary.json \
    --output-dir artifacts/paper1/primary/run
```

Use `--limit`, `--model`, and repeated `--condition` arguments for resource preflight. Each run writes predictions, full traces, summary tables, hashes, configuration, model metadata, environment information, and git revision/dirty status. The `replay` command permits model generation in a separate harness while preserving identical controller and evaluation semantics. The `plot` command renders the machine-readable scaling cells as accuracy curves by model, operand width, condition, and operator count.

9 Limitations

The elicitation instruction asks reflex models to expose an expression followed by an equals sign. This is less explicit than an API call but still changes generation behavior, and a model that answers directly may never trigger. Both trigger rate and conditional-on-trigger accuracy are therefore necessary. The strict lexical grammar does not understand intent and will miss prose arithmetic; expanding semantics belongs to Paper 3. Quote suppression is deliberately limited, and code-like or unusual formatting remains an adversarial surface.

The synthetic task controls arithmetic complexity but not realistic discourse. The textual explicit-tool baseline may understate a model’s native structured function-calling ability. Full-prefix token recomputation overstates runtime cost. Finally, exact calculator output cannot repair an incorrectly copied expression, a model that ignores the result, or a task whose premises are wrong.

10 Related Work

Toolformer trains language models to decide when and how to invoke APIs, including a calculator [1]. PAL delegates model-generated programs to an interpreter [2], while ReAct interleaves model-authored reasoning and actions [3]. These systems establish the value of external execution. Paper 1 isolates a different control point: a runtime recognizes ordinary strict arithmetic already emerging in the output and invokes a bounded engine without a function-name decision. The comparison to explicit invocation is empirical; the paper does not assume that the reflex interface is better.

11 Conclusion

Paper 1 reduces the cognitive-coprocessor proposal to its smallest credible experiment. A character-incremental detector, typed allowlisted micro-IR, exact bounded calculator, append-only state, and immediate text reinjection are enough to test whether automatic computation can improve an autoregressive stream. The implementation is complete at protocol level; model sweeps and replicated analysis remain required before any positive or negative architecture claim.

References

- [1] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023. <https://arxiv.org/abs/2302.04761>.
- [2] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. PAL: Program-aided language models. *ICML*, 2023. <https://arxiv.org/abs/2211.10435>.
- [3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023. <https://arxiv.org/abs/2210.03629>.